

IFT3335 - H2026 (Rachida Toumi : 20171874)

Bang Liu

Avril 2026

1 Introduction

Je voudrais préciser quelques points concernant le notebook. Je l'ai modifié comme suit :

1. J'ai ajouté une fonction pour le cas où la normalisation des données n'est pas autorisée, et je garde les valeurs des pixels entre 0 et 255.
2. Au lieu d'entraîner chaque modèle manuellement, j'ai ajouté une boucle pour entraîner les 10 modèles d'un seul coup, j'ai sauvegardé les métriques pour chaque modèle, afin qu'elles soient tracées plus tard pour l'analyse et la comparaison.
3. J'ai tracé cinq graphiques différents pour les 6 différentes expériences : normalisation des données, nombre de couches, taux d'apprentissage, fonctions de pertes, taille des couches cachées, et taille des minibatches.
4. J'ai fait les analyses et la comparaison des graphiques en me basant sur les questions posées dans la partie 2.
5. J'ai utilisé l'IA de la manière la plus responsable possible, et je suis responsable de chaque ligne de code que j'ai ajoutée. Je suis prête à expliquer, montrer et prouver que je comprends tout ce que j'ai écrit.
6. J'ai utilisé LaTeX pour rédiger ce rapport PDF.

2 Partie 2

2.1 Comprendre la matrice de poids

1. Soit $W \in R^{20 \times 10}$ la matrice de poids, où $W_{j,i}$ représente la connexion du neurone i de la couche l vers le neurone j de la couche $l + 1$. Les lignes impaires de W ont des poids non nuls seulement dans les colonnes 1 à 5, tandis que les lignes paires ont des poids non nuls seulement dans les colonnes 6 à 10 :

$$W_{j,i} \neq 0 \quad \text{si} \quad \begin{cases} j \text{ est impair et } 1 \leq i \leq 5, \\ j \text{ est pair et } 6 \leq i \leq 10, \end{cases}$$

et $W_{j,i} = 0$ autrement.

2. Une image de taille 28×28 donne 784 entrées. Le nombre total de paramètres est :

$$\begin{aligned} & (784 \times 64 + 64) + (64 \times 64 + 64) + (64 \times 10 + 10) \\ & = 50240 + 4160 + 650 = 55050. \end{aligned}$$

Il y a donc 55050 paramètres au total, incluant les biais.

2.2 Normalisation des données

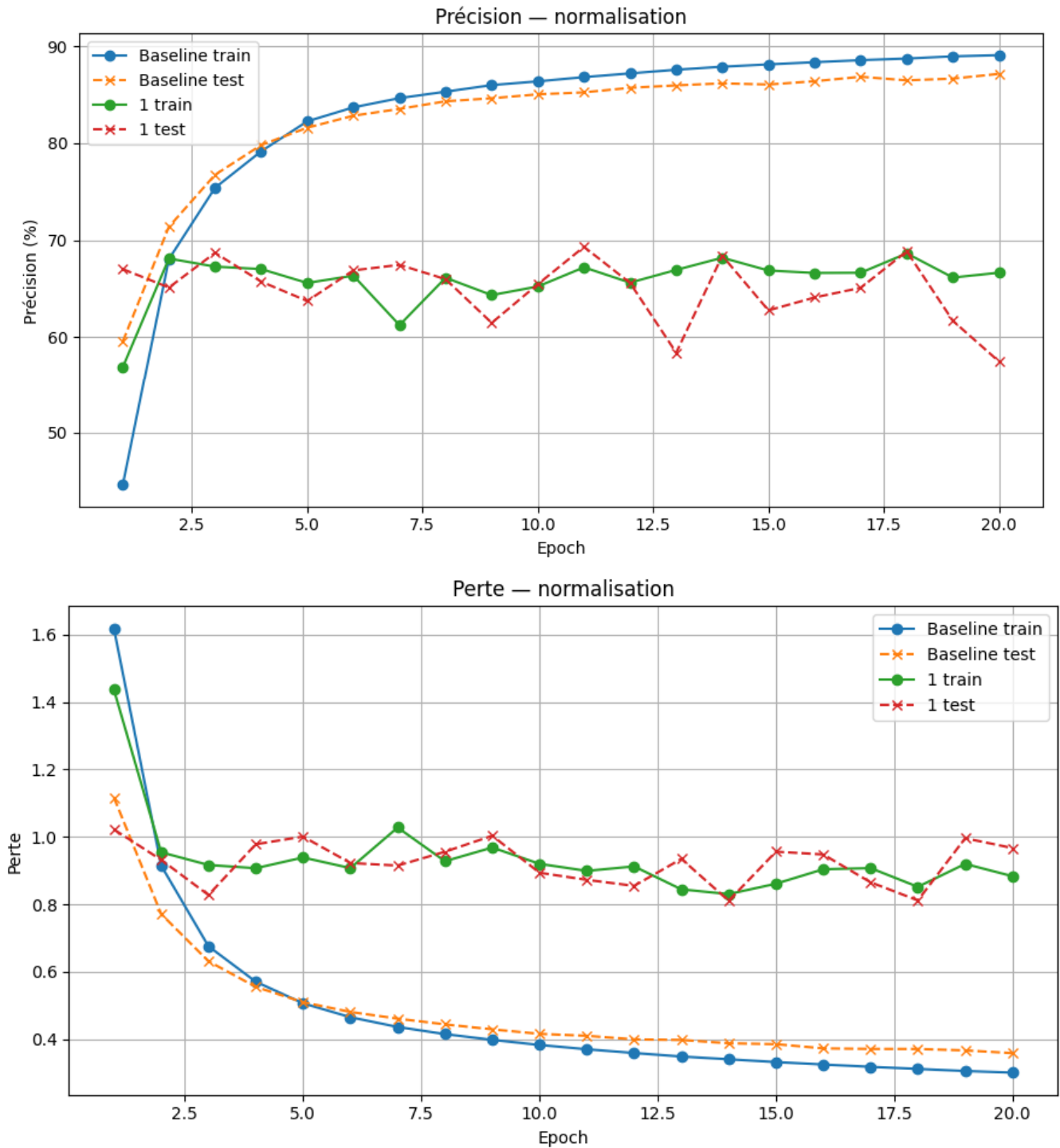


Figure 1:

D'après la figure 1, la normalisation améliore clairement l'apprentissage du modèle. Le modèle **Baseline normalisé** converge de façon stable : sa précision test augmente jusqu'à **87.17%** et sa perte test diminue jusqu'à **0.3590**. Le modèle **non normalisé** est beaucoup plus instable : sa précision test fluctue fortement et termine seulement à **57.47%**, avec une perte test de **0.9670**. Les graphes montrent aussi que, sans normalisation, les courbes train/test ne s'améliorent pas régulièrement. Cela s'explique par le fait que la normalisation place les pixels sur une échelle plus adaptée, ce qui stabilise les gradients et facilite l'optimisation. Donc, dans cette expérience, la normalisation est essentielle pour obtenir une bonne convergence et une meilleure généralisation.

2.3 Nombre de couches

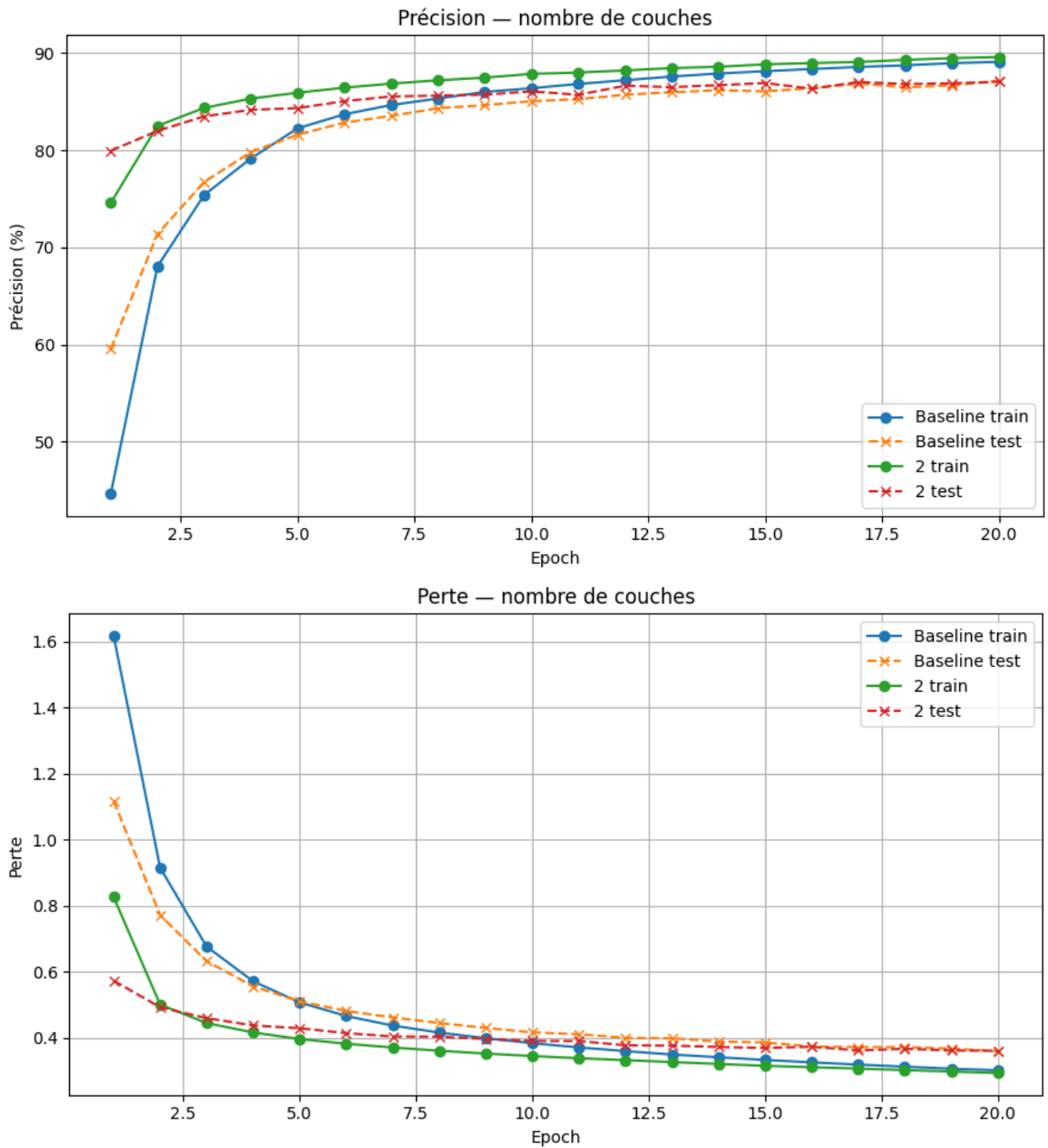


Figure 2:

D'après la figure 2, le modèle avec **1 couche cachée (expérience 2)** performe presque comme le Baseline avec **2 couches cachées**. Le Baseline termine avec **87.17%** de précision test et une perte test de **0.3590**, tandis que l'expérience 2 termine avec **87.06%** et une perte de **0.3605**. Les graphes montrent même que l'expérience 2 apprend plus vite au début, mais les deux modèles convergent vers des performances très proches. Donc, dans ce cas, ajouter une deuxième couche cachée n'améliore pas vraiment la généralisation. Le nombre **70 neurones** a été choisi pour que le modèle à une seule couche ait un nombre de paramètres proche du Baseline. Ainsi, la comparaison mesure surtout l'effet de la profondeur, et non simplement l'effet d'avoir plus ou moins de paramètres.

2.4 Taux d'apprentissage

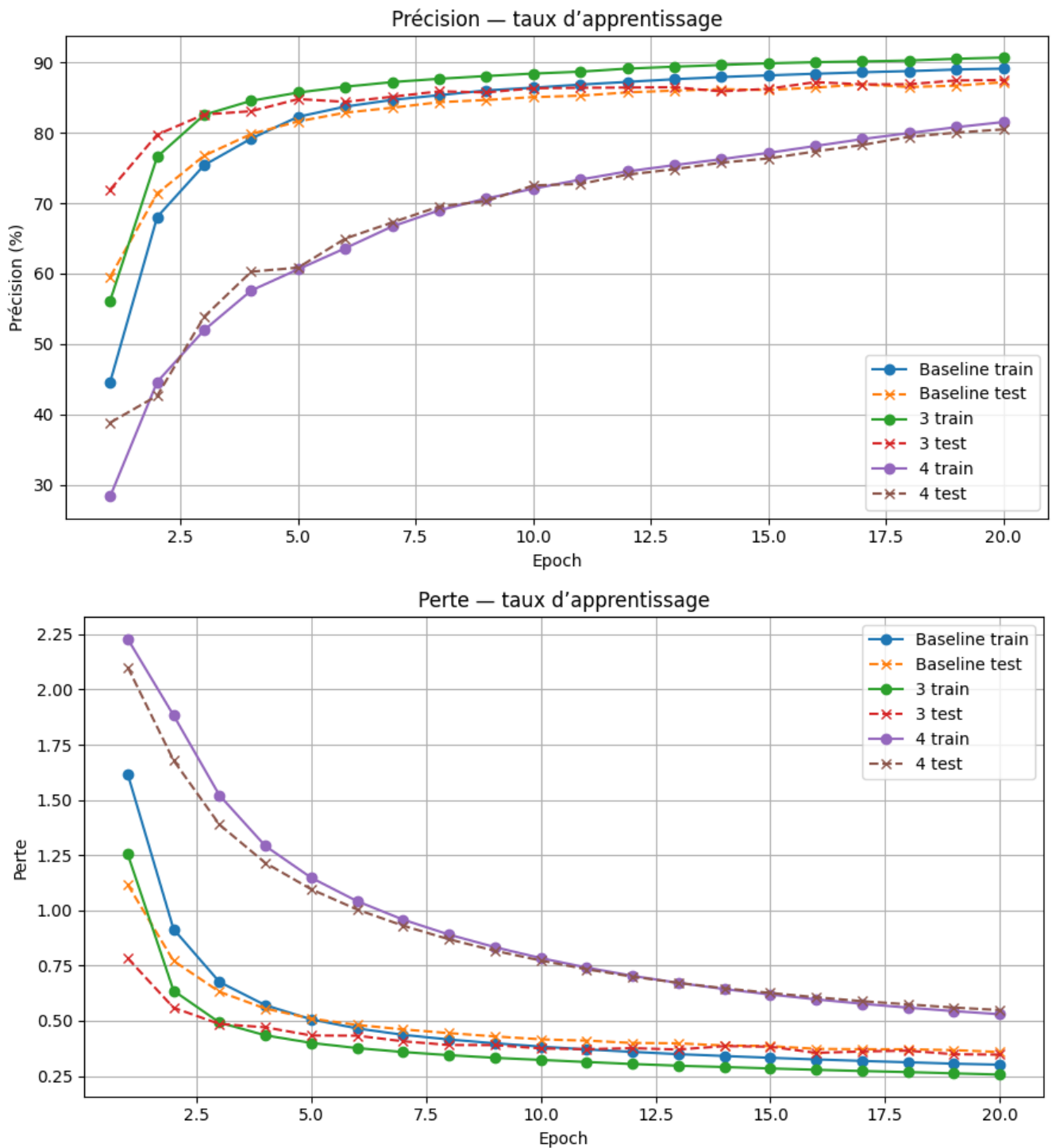


Figure 3:

D'après la figure 3, le taux d'apprentissage influence fortement la vitesse de convergence. Avec $lr = 0.1$, l'expérience 3 apprend plus vite que le Baseline et termine légèrement mieux : **87.49%** de précision test et **0.3470** de perte test. Le Baseline avec $lr = 0.05$ converge aussi bien, mais un peu plus lentement, avec **87.17%** et **0.3590**. L'expérience 4, avec un taux plus petit, apprend beaucoup plus lentement et termine seulement à **80.48%** de précision test. Les graphes montrent que ses pertes restent plus élevées pendant tout l'entraînement. Donc, ici, **0.1** est le meilleur taux parmi les trois, tandis qu'un taux trop faible ralentit l'apprentissage.

2.5 Fonctions de pertes

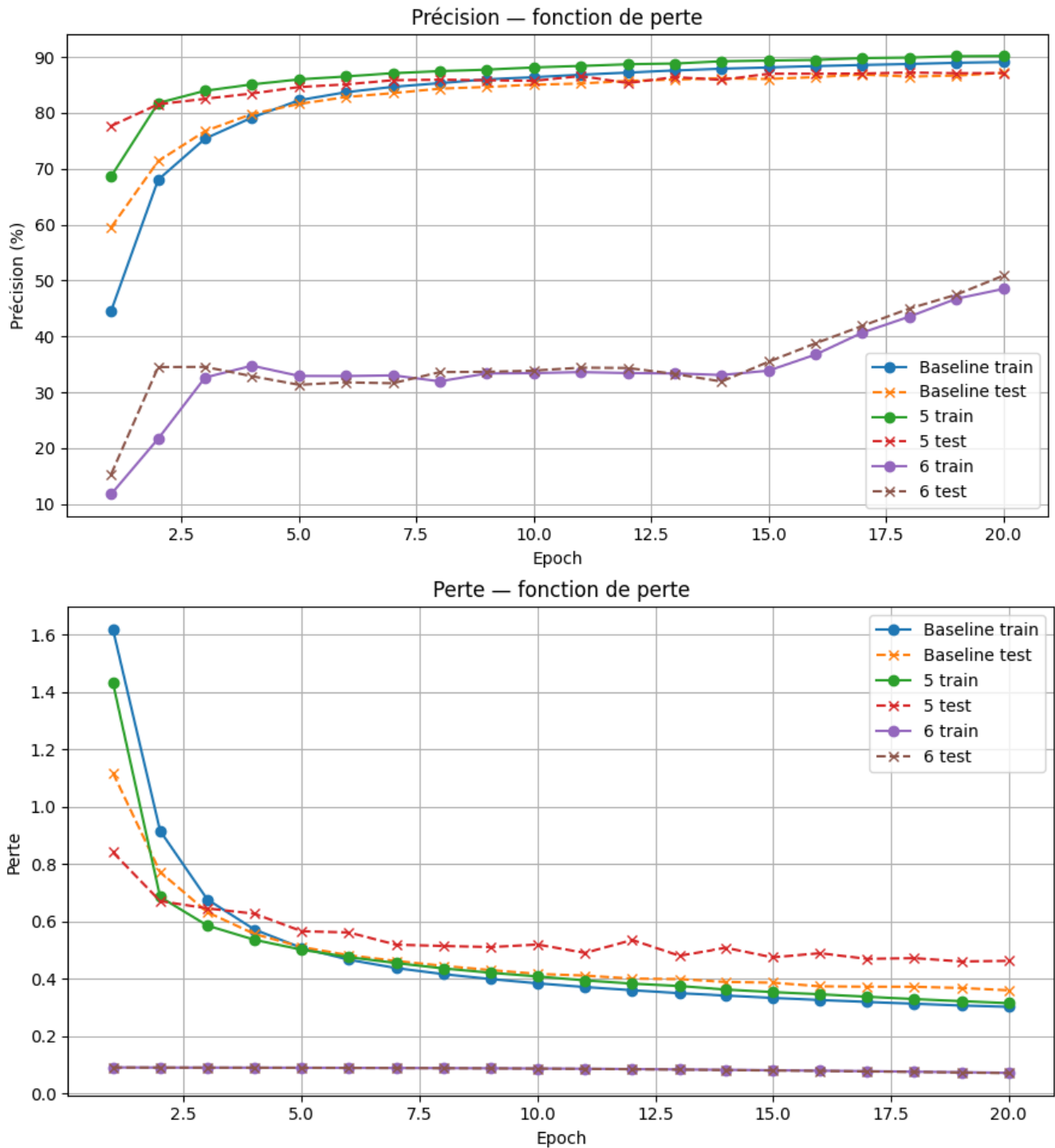


Figure 4:

D'après la figure 4, l'entropie croisée et la perte **Hinge** donnent des performances très proches en précision. Le Baseline avec entropie croisée termine à **87.17%** de précision test avec une perte test de **0.3590**. L'expérience 5 avec Hinge termine presque pareil, à **87.14%**, même si sa perte test est plus élevée **0.4618**, car l'échelle de cette perte est différente. Par contre, l'expérience 6 avec **MSE** fonctionne beaucoup moins bien : elle termine seulement à **50.88%** de précision test. Même si la perte MSE semble numériquement petite, elle n'est pas directement comparable aux autres pertes. Cela montre que la MSE n'est pas bien adaptée ici, car la tâche est une classification multi-classes. Donc, l'entropie croisée reste le choix le plus naturel et le plus fiable pour ce problème.

2.6 Taille des couches cachées

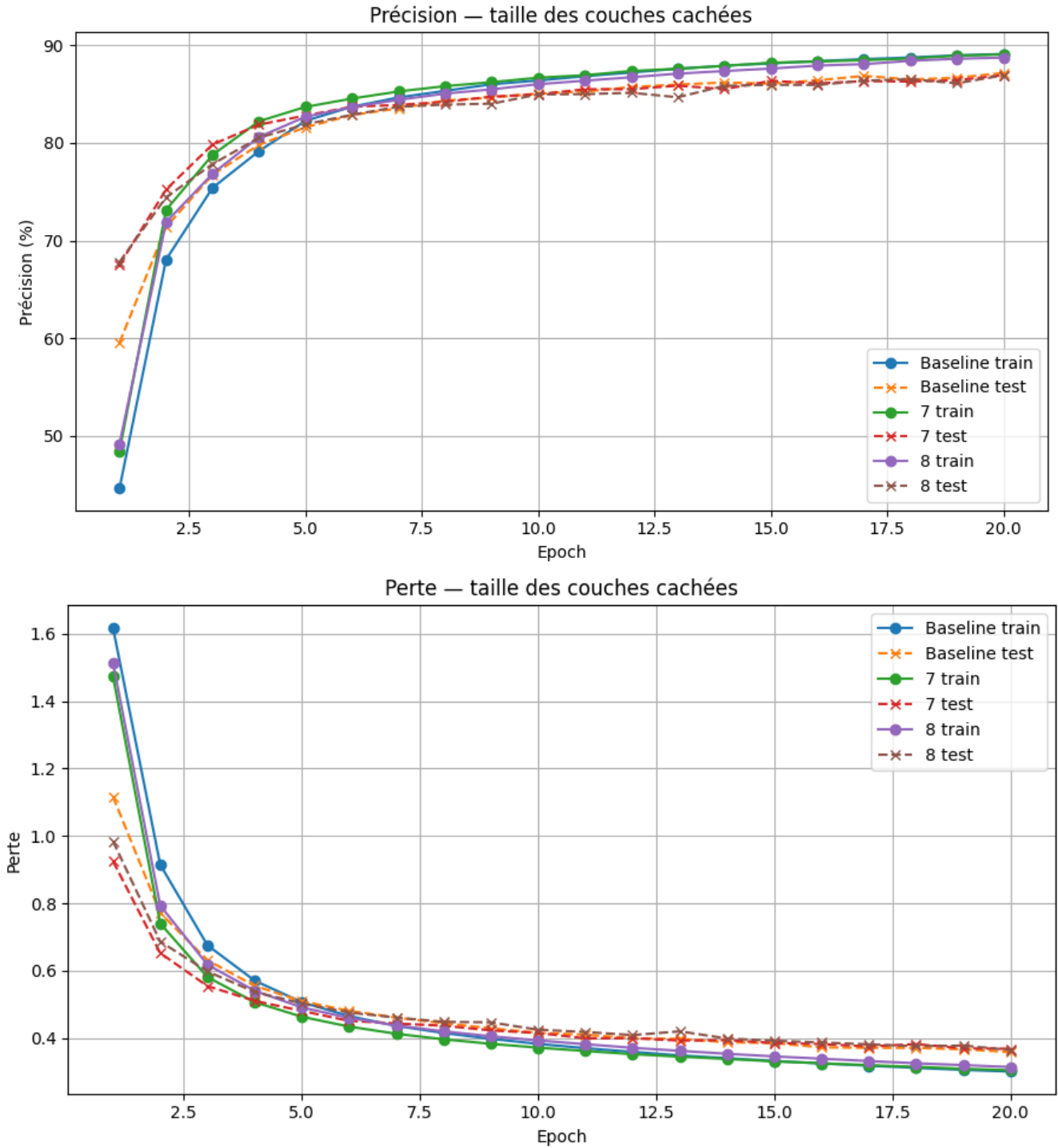


Figure 5:

D'après la figure 5, les trois modèles donnent des performances très proches, donc la taille des couches cachées a un effet limité ici. Le Baseline (**64,64**) termine légèrement mieux avec **87.17%** de précision test et **0.3590** de perte test. L'expérience 7 (**48,300**) termine à **86.95%** avec une perte de **0.3688**, et l'expérience 8 (**300,48**) termine à **86.93%** avec une perte de **0.3652**. Les graphes montrent que les trois modèles convergent de façon stable et presque au même rythme. Le Baseline équilibré reste donc légèrement meilleur, mais l'écart est très petit. Les tailles **48** et **300** ont été choisies pour tester des architectures déséquilibrées : une couche étroite suivie d'une couche large, puis l'inverse. Cela permet d'étudier si l'ordre et la répartition de la capacité entre les couches changent la performance.

2.7 Taille des minibatches

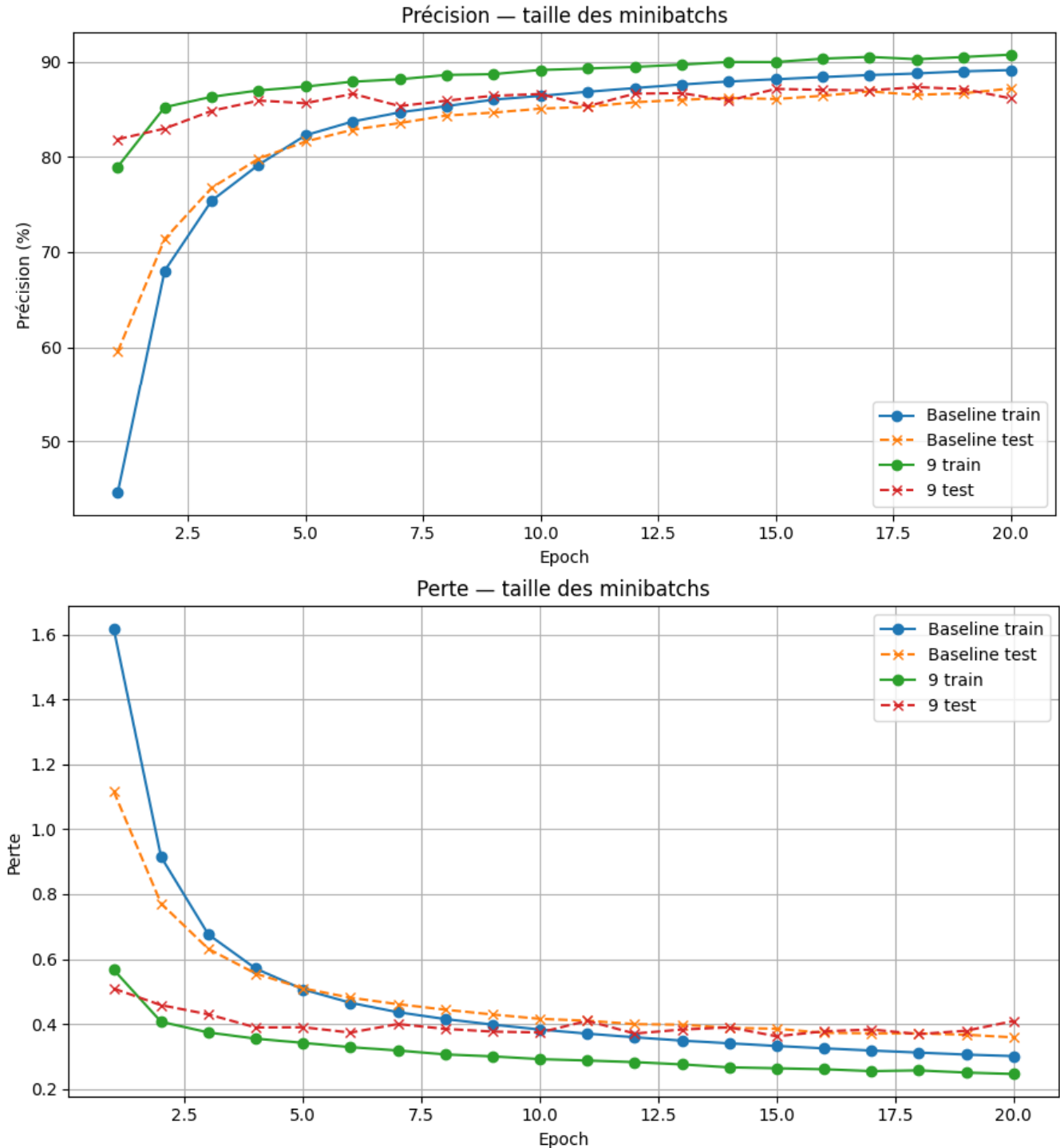


Figure 6:

D'après les métriques d'entraînement et la figure 6, la comparaison est entre le **Baseline batch size = 64** et l'expérience **9 avec batch size = 2**. Le petit minibatch apprend plus vite au début : sa précision test atteint déjà **81.85%** à l'époque 1 contre **59.54%** pour le Baseline. Cependant, il est plus instable : la perte test fluctue davantage et remonte à **0.4094** à la dernière époque. Le Baseline est plus régulier et termine avec une meilleure précision test : **87.17%** contre **86.14%** pour l'expérience 9. Les courbes montrent donc qu'un petit minibatch donne des mises à jour plus fréquentes mais plus bruitées. À l'inverse, un batch size plus grand donne une convergence plus stable et une meilleure généralisation finale ici.